PHD PROGRAM

Applied Data Science & Artificial Intelligence

UNIVERSITÀ DEGLI STUDI DI TRIESTE

Explainable Artificial Intelligence

GRADIENT–BASED METHODS FOR NN



Neural Networks

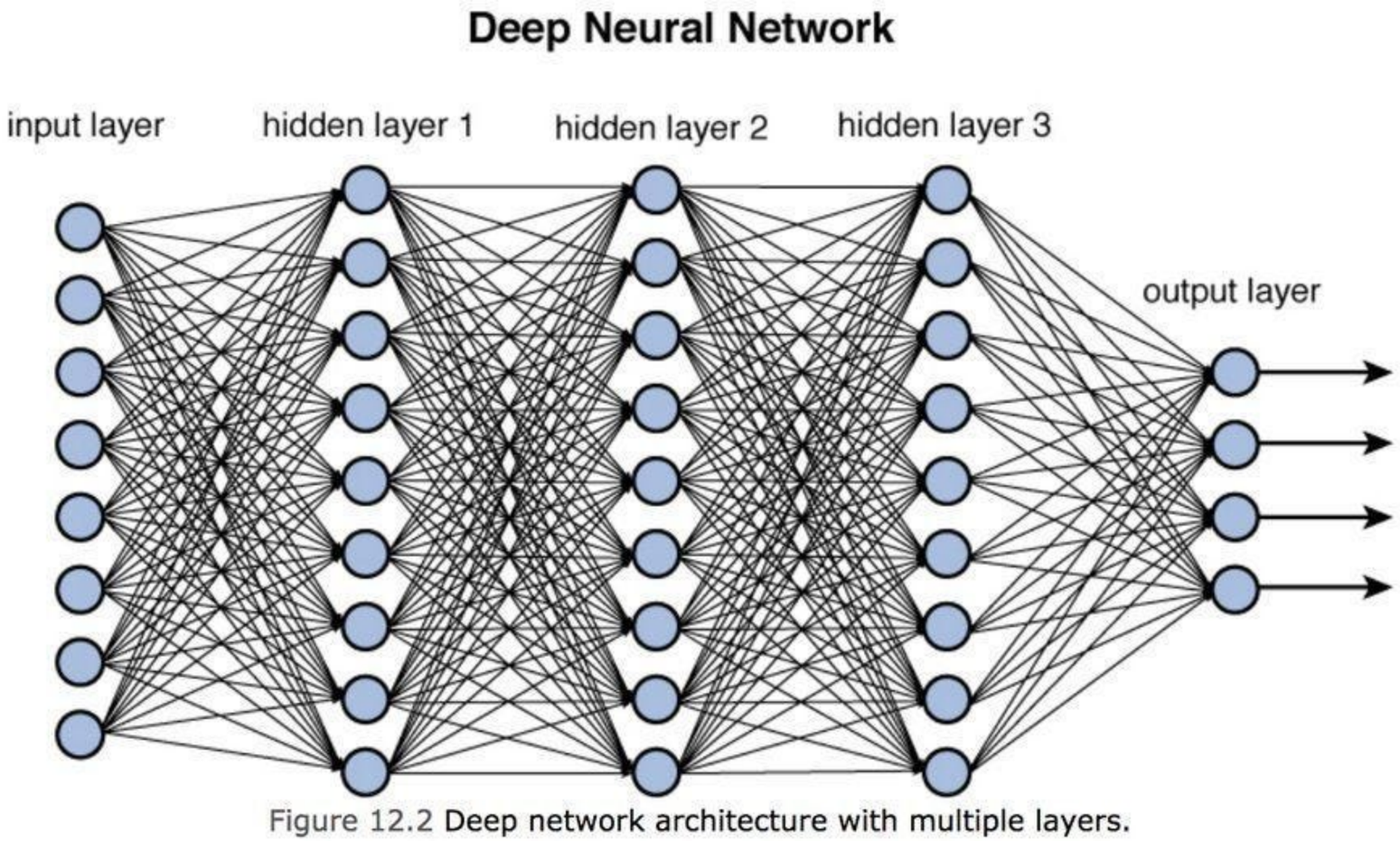


Figure 12.2 Deep network architecture with multiple layers.

Neural networks are

- the most used model
- the most *black box* model

However, the efficient computations implied by their structure allow us to define efficient explanation strategies

Model-specific approaches for Neural Networks

In this lecture:

- DeepLIFT
- Integrated Gradients
- Layer-Wise Relevance Propagation

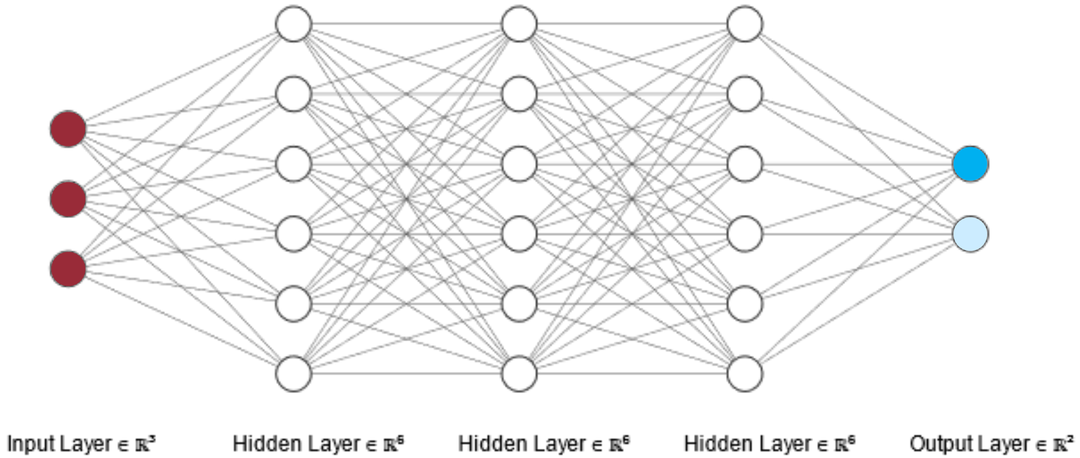
Next time, CNNs specific-methods:

- Class Activation Mapping
- Grad-CAM
- Other extensions

Perturbation-based vs Gradient-based

We can introduce a further distinction in the taxonomy of XAI methods, when specific to NNs.

Perturbation-based: modify the input and compute attributions (model-agnostic)



Gradient-based: follow the inherent structure of the network and use backpropagation to *distribute the gradient* to the input layer

Gradient and Gradient X Input

The simplest neural network-specific attribution method is (trivially) the computation of the **gradient** wrt the input features (not the weights!). The result (for images) is a *heat map* of importances.

$$\nabla_{\mathbf{x}} F(\mathbf{x})$$

A slightly improved version is often referred to as *saliency* and multiplies such **gradients** by the **input value** (rescales the actual relevance, considers the sign, and sets the attribution to zero in case a feature is missing).

$$x_i \nabla_{x_i} F(\mathbf{x})$$

Do you see any issues with these approaches?
Are there better solutions?

DeepLIFT (Deep Learning Important FeaTures)

Learning Important Features Through Propagating Activation Differences

Avanti Shrikumar [1] **Peyton Greenside** [1] **Anshul Kundaje** [1]

Abstract

The purported "black box" nature of neural networks is a barrier to adoption in applications where interpretability is essential. Here we present DeepLIFT (Deep Learning Important FeaTures), a method for decomposing the output prediction of a neural network on a specific input by backpropagating the contributions of all neurons in the network to every feature of the input. DeepLIFT compares the activation of each neuron to its 'reference activation' and assigns contribution scores according to the difference. By optionally giving separate consideration to positive and negative contributions, DeepLIFT can also reveal dependencies which are missed by other approaches. Scores can be computed efficiently in a single backward pass. We apply DeepLIFT to models trained on MNIST and simulated genomic data, and show significant advantages over gradient-based methods. Video tutorial: http://goo.gl/qKb7pL, ICML slides: bit.ly/deeplifticmlslides, ICML talk: https://vimeo.com/238275076, code: http://goo.gl/RM8jvH.

DeepLIFT

DeepLIFT is a **backpropagation-based** approach to explainability: it is more efficient than perturbation-based methods (requires a single backward pass through the net, from a chosen output neuron to the input layer).

It computes feature importances by explaining the difference in output from a *reference output* with the difference in input from a *reference input*.

Let us have t a target output neuron and t^0 the reference activation of t.
Let $x_1,\ x_2,\ \ldots,\ x_n$ be intermediate neurons in the net, necessary and sufficient to compute t.

The *difference-from-reference* is defined as $\Delta t = t\ - t^0$

DeepLIFT

The **contribution score** $C_{\Delta x_i \Delta t}$ of Δx_i satisfies the summation-to-delta property: $\sum_{i=1}^{n} C_{\Delta x_i \Delta t} = \Delta \mathrm{t}$

A **multiplier** is defined as $m_{\Delta x_i \Delta t} = \frac{C_{\Delta x_i \Delta t}}{\Delta x_i}$ and represents the contribution of Δx_i to $\Delta \mathrm{t}$, divided by Δx_i (analogy with partial derivatives).

Having $x_1, x_2, \ldots, x_n$ input layer neurons, $y_1, \ldots, y_n$ hidden layer neurons and t a target output neuron, the multipliers satisfy the following chain rule:

$$m_{\Delta x_i \Delta t} = \sum_j m_{\Delta x_i \Delta y_j} \cdot m_{\Delta y_j \Delta t}$$

The chain rule allows for efficient computation via backpropagation.

Reference and rules

The reference of a neuron is defined as its *activation value on a reference input*.
Choices of the reference are domain-specific, generally a zero-vector could work but one may choose to compute the attributions against multiple reference values and then average.
For MNIST, a good reference is a black image (colour of the background for the digits).

DeepLIFT introduces the difference between positive and negative contributions.

$$\Delta x = \Delta x^+ + \Delta x^- \qquad \Delta y = \Delta y^+ + \Delta y^-$$
$$C_{\Delta x \Delta y} = C_{\Delta x^+ \Delta y} + C_{\Delta x^- \Delta y}$$

This allows $m_{\Delta x^+ \Delta y} != m_{\Delta x^- \Delta y}$ but for Linear and Rescale rules (next slide) they coincide.

Rules

The rules assign contribution scores for each neuron as a function of its inputs. Cconsidering also the chain rule for multipliers, allows us to backpropagate till the input layer.

Linear Rule (Dense and Convolutional Layers) x_i input neurons of y:

$$y = b + \sum_i w_i x_i \qquad \Delta y = \sum_i w_i \Delta x_i$$

$$\begin{aligned}\Delta y^+ &= \sum_i 1\{w_i \Delta x_i > 0\} w_i \Delta x_i \\ &= \sum_i 1\{w_i \Delta x_i > 0\} w_i (\Delta x_i^+ + \Delta x_i^-)\end{aligned}$$

$$\begin{aligned}\Delta y^- &= \sum_i 1\{w_i \Delta x_i < 0\} w_i \Delta x_i \\ &= \sum_i 1\{w_i \Delta x_i < 0\} w_i (\Delta x_i^+ + \Delta x_i^-)\end{aligned}$$

$$\begin{aligned}C_{\Delta x_i^+ \Delta y^+} &= 1\{w_i \Delta x_i > 0\} w_i \Delta x_i^+ \\ C_{\Delta x_i^- \Delta y^+} &= 1\{w_i \Delta x_i > 0\} w_i \Delta x_i^- \\ C_{\Delta x_i^+ \Delta y^-} &= 1\{w_i \Delta x_i < 0\} w_i \Delta x_i^+ \\ C_{\Delta x_i^- \Delta y^-} &= 1\{w_i \Delta x_i < 0\} w_i \Delta x_i^-\end{aligned}$$

$$m_{\Delta x_i^+ \Delta y^+} = m_{\Delta x_i^- \Delta y^+} = 1\{w_i \Delta x_i > 0\} w_i$$

$$m_{\Delta x_i^+ \Delta y^-} = m_{\Delta x_i^- \Delta y^-} = 1\{w_i \Delta x_i < 0\} w_i.$$

Rules

Rescale Rule (nonlinear transformations that take a single input, like ReLU)
$y = f(x)$

$$C_{\Delta x\,\Delta y} = \Delta y \qquad m_{\Delta x\,\Delta y} = \frac{\Delta y}{\Delta x}$$

$$\Delta y^+ = \frac{\Delta y}{\Delta x}\Delta x^+ = C_{\Delta x^+ \Delta y^+}$$
$$\Delta y^- = \frac{\Delta y}{\Delta x}\Delta x^- = C_{\Delta x^- \Delta y^-}$$

$$m_{\Delta x^+ \Delta y^+} = m_{\Delta x^- \Delta y^-} = m_{\Delta x \Delta y} = \frac{\Delta y}{\Delta x}$$

Results from the paper

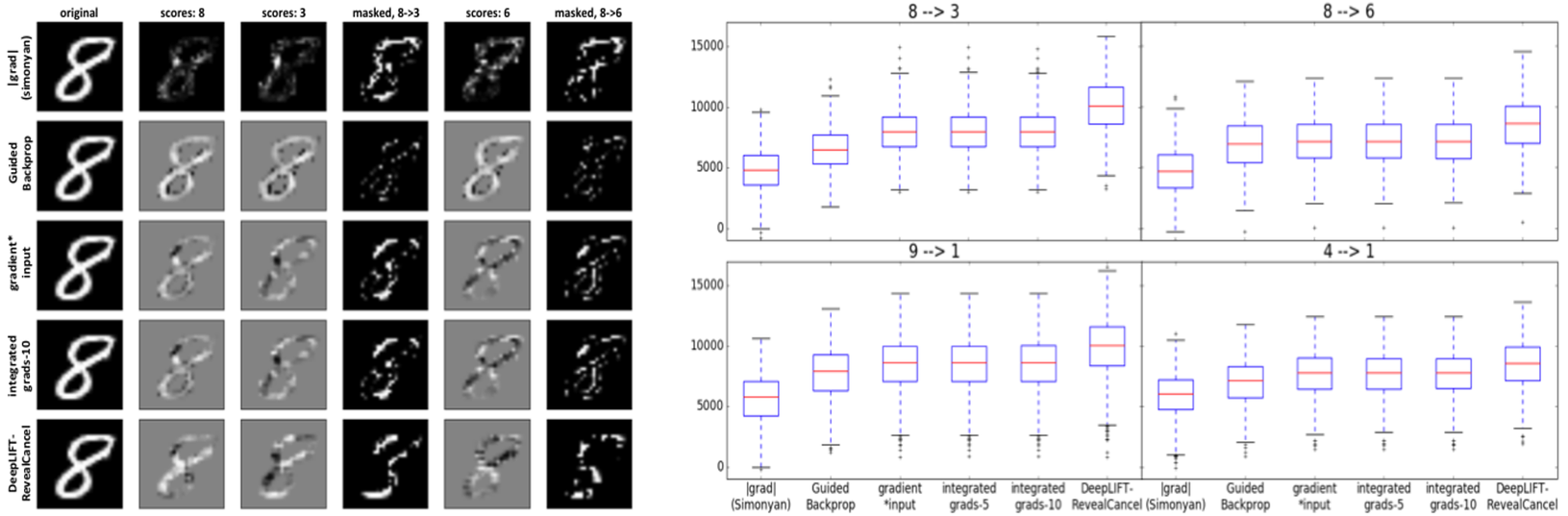


Figure 4. **DeepLIFT with the RevealCancel rule better identifies pixels to convert one digit to another.** Left result of masking pixels ranked as most important for the original class (8) relative to the target class (3 or 6). Importance scores for class 8, 3 and 6 are also shown. The selected image had the highest change in log-odds scores for the 8→6 conversion using gradient*input or integrated gradients to rank pixels. Right : boxplots of increase in log-odds scores of target vs. original class after the mask is applied, for 1K images belonging to the original class in the testing set. "Integrated gradients-n" refers to numerically integrating the gradients over n evenly-spaced intervals using the midpoint rule.

Something familiar

DeepLIFT is linked to the model-specific implementation of SHAP devoted to neural networks (DeepSHAP), as presented in the original SHAP paper.

DeepLIFT can be considered as a fast approximation of Shapley values, under the assumptions that the input features are independent and that the deep model is linear. The DeepLIFT multipliers are re-expressed in terms of Shapley values and passed backward through the net.

Note that Lundberg and Lee cited a pre-print version of DeepLIFT, in which only the Linear and Rescale rules were present, not the separation between positive and negative contributions.

Integrated Gradients

Axiomatic Attribution for Deep Networks

Mukund Sundararajan [*1] **Ankur Taly** [*1] **Qiqi Yan** [*1]

Abstract

We study the problem of attributing the prediction of a deep network to its input features, a problem previously studied by several other works. We identify two fundamental axioms—*Sensitivity* and *Implementation Invariance* that attribution methods ought to satisfy. We show that they are not satisfied by most known attribution methods, which we consider to be a fundamental weakness of those methods. We use the axioms to guide the design of a new attribution method called *Integrated Gradients*. Our method requires no modification to the original network and is extremely simple to implement; it just needs a few calls to the standard gradient operator. We apply this method to a couple of image models, a couple of text models and a chemistry model, demonstrating its ability to debug networks, to extract rules from a network, and to enable users to engage with models better.

Axioms

Sensitivity(a): *if an input and a baseline differ in one feature but have different predictions, then the differing feature should be given a non-zero attribution.*

Implementation invariance: *the attributions are always identical for two functionally equivalent networks (i.e. their outputs are equal for all inputs).*

It can be shown that common feature attributions methods in literature break at least one of these two axioms. Integrated Gradients instead is built with them in mind.

Integrated Gradients

Let us have $F: R^n \rightarrow [0,1]$ a function representing a deep network and $x, x' \in R^n$ the chosen input point and baseline respectively.

Integrated Gradients are defined as the path integral of the gradients along the straight-line path from the baseline x' to the input x.

$$IntegratedGradients_i(x) = \left(x_i - x_i'\right) \times \int_{\alpha=0}^{1} \frac{\partial F(x' + \alpha \cdot (x - x'))}{\partial x_i} d\alpha$$

Both axioms are satisfied.

Integrated Gradients

The baseline should convey a complete absence of signal, to avoid introducing undesirable artifacts. For images, a black image. For text, an all-zero text embedding.

Integrated Gradients are practically computed via the following approximation, with $m \in (20,300)$

$$IntegratedGradients_i^{\text{approx}}(x) = \left(x_i - x_i'\right) \times \sum_{k=1}^{m} \frac{\partial F(x' + \frac{k}{m} \cdot (x - x'))}{\partial x_i} \times \frac{1}{m}$$

Straight-line path from the baseline to the input

if a NN does not depend on x,
x has zero attribution

attribution (f+g) = attribution(f) + attribution(g)

Attribution methods based on path integrated gradients are part of the class of *path methods*. They satisfy desirable axioms (such as implementation invariance, sensitivity$_{(b)}$, linearity and completeness).

sum of attributions = F(x) - F(baseline)

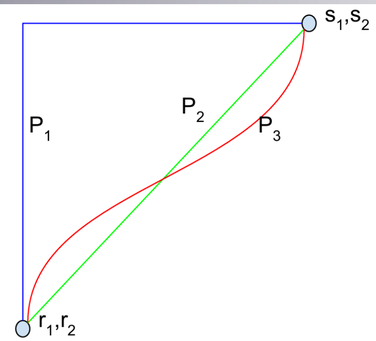


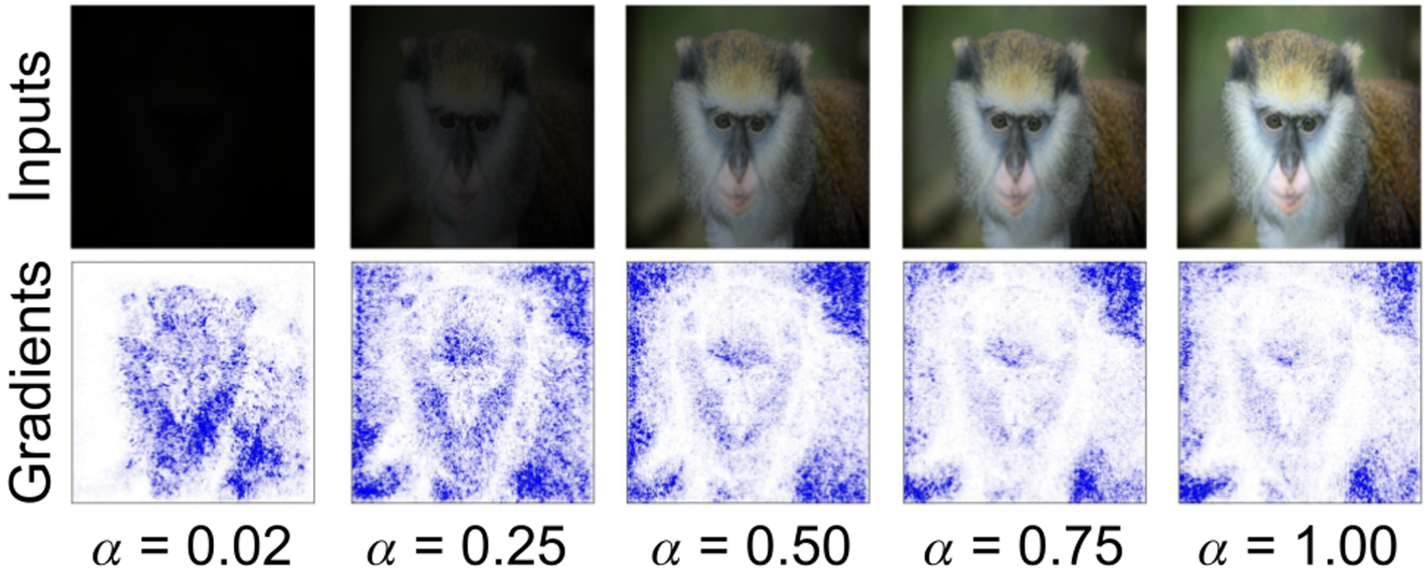
Figure 1. Three paths between an a baseline (r_1, r_2) and an input (s_1, s_2). Each path corresponds to a different attribution method. The path P_2 corresponds to the path used by integrated gradients.

Integrated Gradients is the only path method which is symmetry preserving (*two input variables are symmetric wrt a function if swapping them does not change the function*): if a function is symmetric w.r.t variables x and y, and those features have the same values for the input and the baseline, they will have the same attribution.

Note that path I.G. comes from cost-sharing literature in economics: Integrated Gradients uses a cost-sharing method called Aumann-Shapley (1974) while SHAP averages over multiple paths with the Shapley-Shubik method (1971).

Straight-line path from the baseline to the input

When dealing with images, the straight-line path from baseline (black image) to the input translates into a series of increasingly more luminous images.



Results from the paper

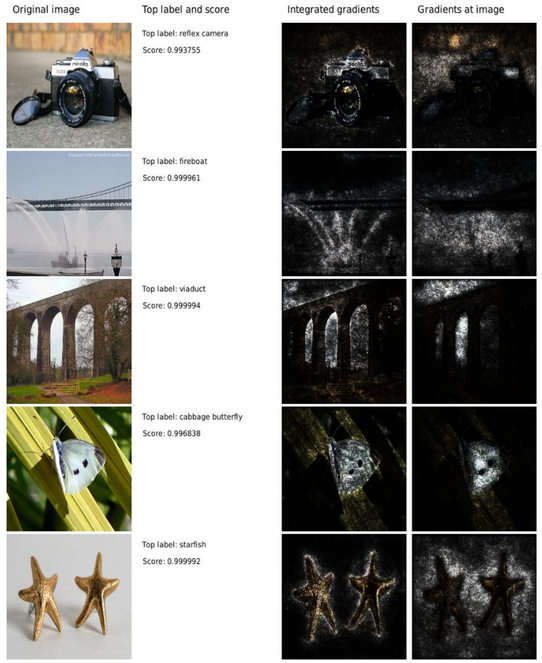


Figure 2. **Comparing integrated gradients with gradients at the image.** Left-to-right: original input image, label and softmax score for the highest scoring class, visualization of integrated gradients, visualization of gradients*image. Notice that the visualizations obtained from integrated gradients are better at reflecting distinctive features of the image.

how many townships have a population above 50 ? [prediction: NUMERIC]
what is the difference in population between fora and masilo [prediction: NUMERIC]
how many athletes are not ranked ? [prediction: NUMERIC]
what is the total number of points scored ? [prediction: NUMERIC]
which film was before the audacity of democracy ? [prediction: STRING]
which year did she work on the most films ? [prediction: DATETIME]
what year was the last school established ? [prediction: DATETIME]
when did ed sheeran get his first number one of the year ? [prediction: DATETIME]
did charles oakley play more minutes than robert parish ? [prediction: YESNO]

Figure 4. **Attributions from question classification model.** Term color indicates attribution strength—Red is positive, Blue is negative, and Gray is neutral (zero). The predicted class is specified in square brackets.

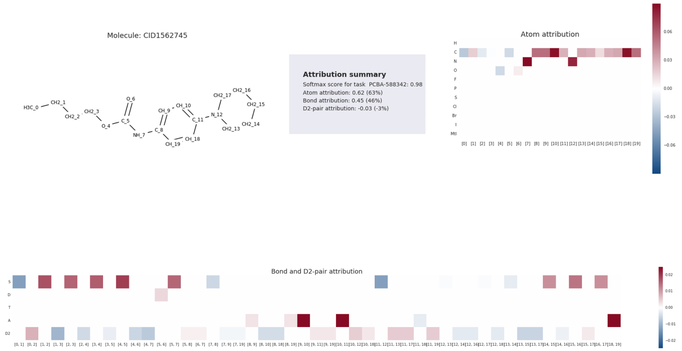


Figure 6. **Attribution for a molecule under the W2N2 network (Kearnes et al., 2016).** The molecules is active on task PCBA-58432.

Layer-Wise Relevance Propagation

On Pixel-Wise Explanations for Non-Linear Classifier Decisions by Layer-Wise Relevance Propagation

Sebastian Bach[1,2☯]*, Alexander Binder[2,5☯], Grégoire Montavon[2], Frederick Klauschen[3], Klaus-Robert Müller[2,4]*, Wojciech Samek[1,2]*

1 Machine Learning Group, Fraunhofer Heinrich Hertz Institute, Berlin, Germany, **2** Machine Learning Group, Technische Universität Berlin, Berlin, Germany, **3** Charité University Hospital, Berlin, Germany, **4** Department of Brain and Cognitive Engineering, Korea University, Seoul, Korea, **5** ISTD Pillar, Singapore University of Technology and Design (SUTD), Singapore

☯ These authors contributed equally to this work.
* sebastian.bach@hhi.fraunhofer.de (SB), klaus-robert.mueller@tu-berlin.de (KM), wojciech.samek@hhi.fraunhofer.de (WS)

Abstract

Understanding and interpreting classification decisions of automated image classification systems is of high value in many applications, as it allows to verify the reasoning of the system and provides additional information to the human expert. Although machine learning methods are solving very successfully a plethora of tasks, they have in most cases the disadvantage of acting as a black box, not providing any information about what made them arrive at a particular decision. This work proposes a general solution to the problem of understanding classification decisions by pixel-wise decomposition of nonlinear classifiers. We introduce a methodology that allows to visualize the contributions of single pixels to predictions for kernel-based classifiers over Bag of Words features and for multilayered neural networks. These pixel contributions can be visualized as heatmaps and are provided to a human expert who can intuitively not only verify the validity of the classification decision, but also focus further analysis on regions of potential interest. We evaluate our method for classifiers trained on PASCAL VOC 2009 images, synthetic image data containing geometric shapes, the MNIST handwritten digits data set and for the pre-trained ImageNet model available as part of the Caffe open source package.

Layer-Wise Relevance Propagation

Layer-Wise Relevance Propagation backpropagates the output score (or *relevance*) $f(x)$ through the net to the input layer, aided by carefully designed *propagation rules*.

The relevances satisfy the conservation rule, for which each neuron redistributes exactly the same amount of relevance it has received.

Having R_j the relevance for neuron j , for each pair of layers it holds $\sum_j R_j = \sum_k R_k$, hence globally for each layer it holds $\sum_i R_i = f(x)$.

Layer-Wise Relevance Propagation

The conservation property does not guarantee a unique, nor a meaningful, solution.

The following property is introduced: the relevance of a neuron is equal to the sum of *messages* coming from interconnected neurons of the following layer.

$$R_i^{(l)} = \sum_{k:i \text{ is input for neuron } k} R_{i\leftarrow k}^{(l,\,l+1)}$$

For the output layer, $R_i^{\text{out}} = f(x)$

Also, to satisfy the conservation property, it can be proven that $R_k^{(l+1)} = \sum_{i:i \text{ is input for neuron } k} R_{i\leftarrow k}^{(l,\,l+1)}$

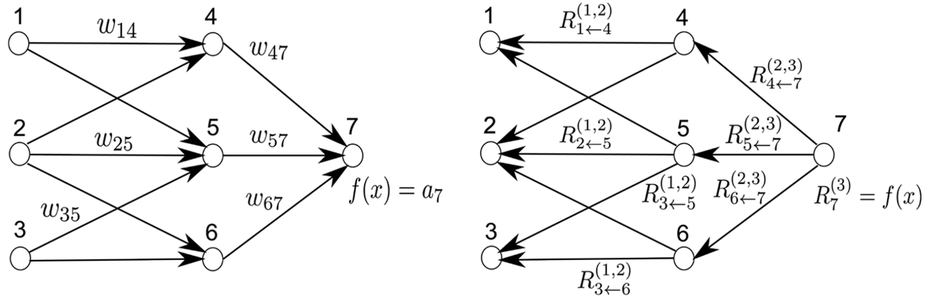


Fig 2. Left: A neural network-shaped classifier during prediction time. w_{ij} are connection weights. a_i is the activation of neuron *i*. Right: The neural network-shaped classifier during layer-wise relevance computation time. $R_i^{(l)}$ is the relevance of neuron *i* which is to be computed. In order to facilitate the computation of $R_i^{(l)}$ we introduce messages $R_{i\leftarrow j}^{(l,l+1)}$. $R_{i\leftarrow j}^{(l,l+1)}$ are messages which need to be computed such that the layer-wise relevance in Eq (2) is conserved. The messages are sent from a neuron *i* to its input neurons *j* via the connections used for classification, e.g. 2 is an input neuron for neurons 4, 5, 6. Neuron 3 is an input neuron for 5, 6. Neurons 4, 5, 6 are the input for neuron 7.

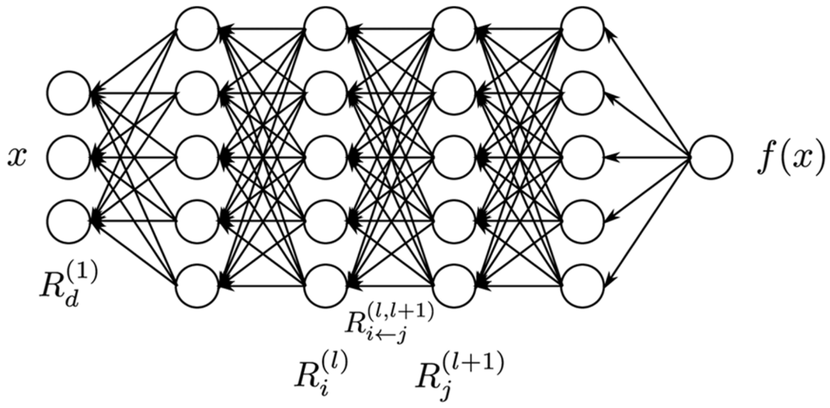
Layer-Wise Relevance Propagation

For a non-linear layer of the form $y_i = \sigma(\sum_i x_i w_{ij} + b_j) = \sigma\left(\sum_i z_{ij} + b_j\right) = \sigma(z_j)$ we can compute the relevances as $R_{i\leftarrow j}^{(l,\,l+1)} = \frac{z_{ij}}{z_j} R_j^{(l+1)}$

This definition approximately satisfies the conservation property of layer relevances:

$$\sum_{i\in l} R_{i\leftarrow j}^{(l,\,l+1)} = R_j^{(l+1)}(1 - \frac{b_j}{z_j})$$

Where the multiplier accounts for the relevance that is absorbed (or injected) by the bias term.



LRP rules for RELU activation functions

The neurons with RELU activation: $a_k = \max(0, \sum_{0,j} a_j w_{jk})$ where the sum runs over all lower-layer activations.

$$\textbf{LRP-0}: R_j = \sum_k \frac{a_j w_{jk}}{\sum_{0,j} a_j w_{jk}} R_k$$

A uniform application of this rule to the whole NN produces an explanation equivalent to GradientXInput.

$$\textbf{LRP-}\epsilon: R_j = \sum_k \frac{a_j w_{jk}}{\epsilon + \sum_{0,j} a_j w_{jk}} R_k$$

$\epsilon > 0$ is added for stability. When it becomes larger, only the most important explanations *survive*.

$$\textbf{LRP-}\gamma: R_j = \sum_k \frac{a_j \cdot (w_{jk} + \gamma w^+_{jk})}{\sum_{0,j} a_j \cdot (w_{jk} + \gamma w^+_{jk})} R_k{}^*$$

γ controls by how much positive attributions are favoured.
To deal asymmetrically with positive and negative attributions, see **LRP-$\alpha\beta$** rule.
Note that with $\gamma \to \infty$ LRP- γ is equivalent to LPR- $\alpha_1\beta_0$

$*$ w^+_{jk} is a positive weight

Efficient Implementation of LRP

Having a generic rule $R_j = \sum_k \frac{a_j \cdot \rho(w_{jk})}{\epsilon + \sum_{0,j} a_j \cdot \rho(w_{jk})} R_k$, the computation of the propagation follows:

$$\forall k : z_k = \epsilon + \sum_{0,j} a_j \cdot \rho\left(w_{jk}\right) \qquad \text{(forward pass)}$$

$$\forall k : s_k = R_k / z_k \qquad \text{(element-wise division)}$$

$$\forall j : c_j = \sum_k \rho\left(w_{jk}\right) \cdot s_k \qquad \text{(backward pass)}$$

$$\forall j : R_j = a_j c_j \qquad \text{(element-wise product)}$$

```
def relprop(a,layer,R):
    z = epsilon + rho(layer).forward(a)
    s = R/(z+1e-9)
    (z*s.data).sum().backward()
    c = a.grad
    R = a*c
    return R
```

$$c_j = \left[\nabla\left(\sum_k z_k(\boldsymbol{a}) \cdot s_k\right)\right]_j$$

LRP as Deep Taylor Decomposition

Deep Taylor Decomposition applies a Taylor expansion at each layer. For relevance $R_k(\mathbf{a})$:

$$R_k(\boldsymbol{a}) = R_k(\widetilde{\boldsymbol{a}}) + \sum_{0,j}(a_j - \widetilde{a}_j) \cdot [\nabla R_k(\widetilde{\boldsymbol{a}})]_j + \dots$$

The generic form of $R_k(\mathbf{a})$ is not known. We can define a surrogate model as:

$$\widehat{R}_k(\boldsymbol{a}) = \max\left(0, \sum_{0,j} a_j w_{jk}\right) \cdot c_k.$$

whose DTD is $\widehat{R}_k(\boldsymbol{a}) = \widehat{R}_k(\widetilde{\boldsymbol{a}}) + \sum_{0,j}(a_j - \widetilde{a}_j) \cdot w_{jk}\, c_k.$

By choosing the reference point $\tilde{a}$ appropriately, we recover 0, ε and γ rules.

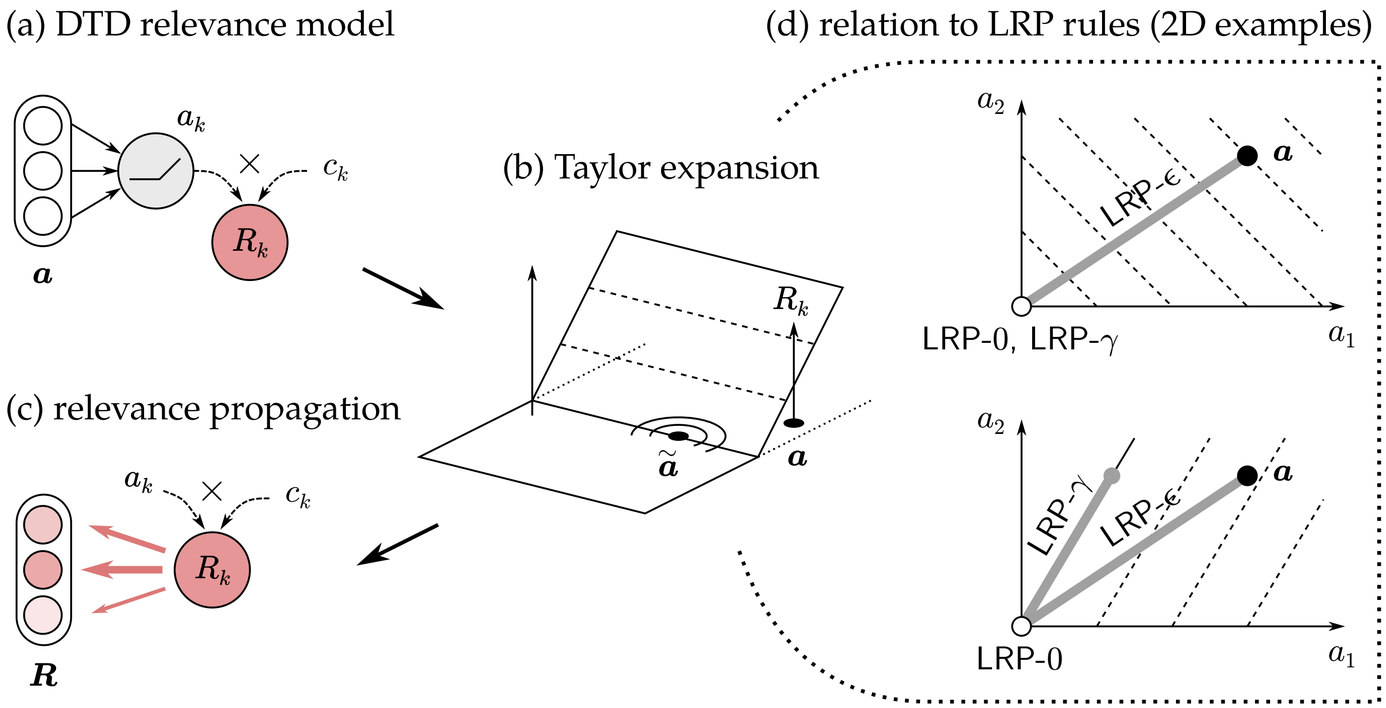
LRP as Deep Taylor Decomposition

$$\widehat{R}_k(\boldsymbol{a}) = \widehat{R}_k(\widetilde{\boldsymbol{a}}) + \sum\nolimits_{0,j}(a_j - \widetilde{a}_j) \cdot w_{jk}\, c_k.$$

LRP-γ: Intersection between the RELU hinge and the line: $\{\boldsymbol{a} - t \cdot \boldsymbol{a} \odot (\mathbf{1} + \gamma \cdot \mathbf{1}_{\boldsymbol{w}_k \succeq \mathbf{0}}) \mid t \in \mathbb{R}\}$.

LRP-0: $\widetilde{\boldsymbol{a}} = \mathbf{0}$.

LRP-ε: $\widetilde{\boldsymbol{a}} = \epsilon \cdot (a_k + \epsilon)^{-1}\, \boldsymbol{a}$.



Different Rules for Different Layers

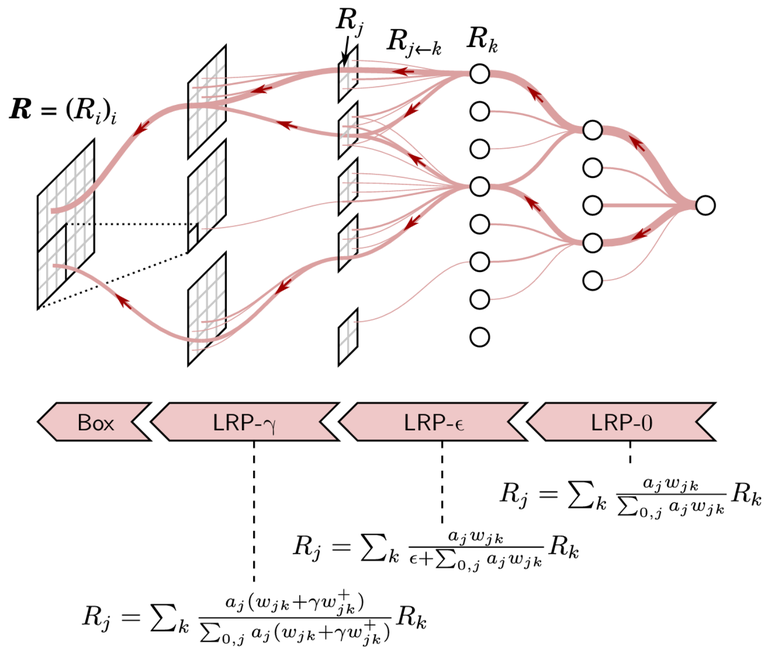


Fig. 4. Illustration of the LRP propagation procedure applied to a neural network. The prediction at the output is propagated backward in the network, using various propagation rules, until the input features are reached. The propagation flow is shown in red.

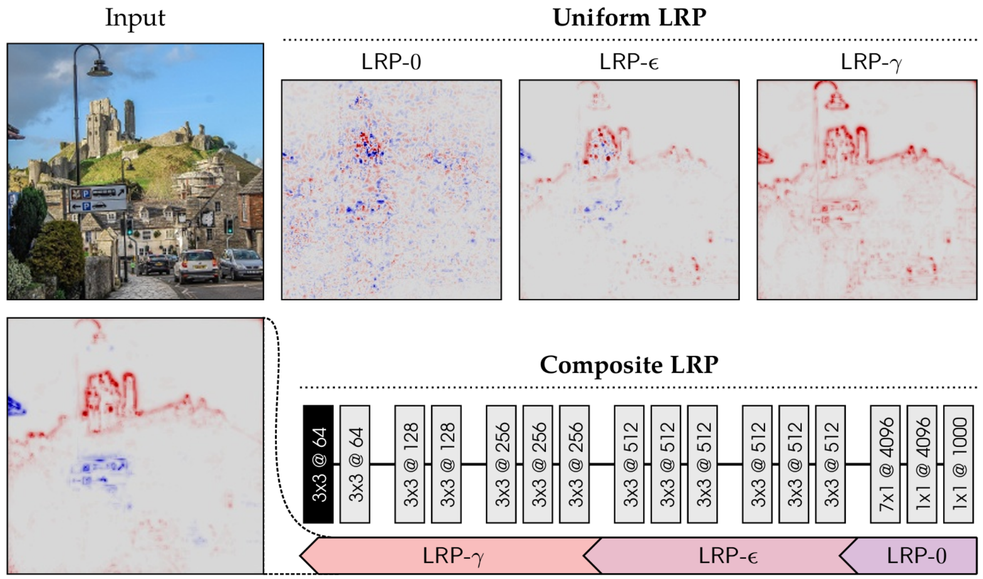


Fig. 10.4. Input image and pixel-wise explanations of the output neuron 'castle' obtained with various LRP procedures. Parameters are $\epsilon = 0.25\,\text{std}$ and $\gamma = 0.25$.

Different Rules for Different Layers

Name	Formula	Usage	DTD
LRP-0 [7]	$R_j = \sum_k \frac{a_j w_{jk}}{\sum_{0,j} a_j w_{jk}} R_k$	upper layers	✓
LRP-ϵ [7]	$R_j = \sum_k \frac{a_j w_{jk}}{\epsilon + \sum_{0,j} a_j w_{jk}} R_k$	middle layers	✓
LRP-γ	$R_j = \sum_k \frac{a_j (w_{jk} + \gamma w_{jk}^+)}{\sum_{0,j} a_j (w_{jk} + \gamma w_{jk}^+)} R_k$	lower layers	✓
LRP-$\alpha\beta$ [7]	$R_j = \sum_k \left(\alpha \frac{(a_j w_{jk})^+}{\sum_{0,j} (a_j w_{jk})^+} - \beta \frac{(a_j w_{jk})^-}{\sum_{0,j} (a_j w_{jk})^-} \right) R_k$	lower layers	$\times^\star$
flat [30]	$R_j = \sum_k \frac{1}{\sum_j 1} R_k$	lower layers	$\times$
w^2-rule [36]	$R_i = \sum_j \frac{w_{ij}^2}{\sum_i w_{ij}^2} R_j$	first layer ($\mathbb{R}^d$)	✓
$z^\mathcal{B}$-rule [36]	$R_i = \sum_j \frac{x_i w_{ij} - l_i w_{ij}^+ - h_i w_{ij}^-}{\sum_i x_i w_{ij} - l_i w_{ij}^+ - h_i w_{ij}^-} R_j$	first layer (pixels)	✓

(* DTD interpretation only for the case $\alpha = 1, \beta = 0$.)

* DTD: Deep Taylor Decompositon

Results from the paper

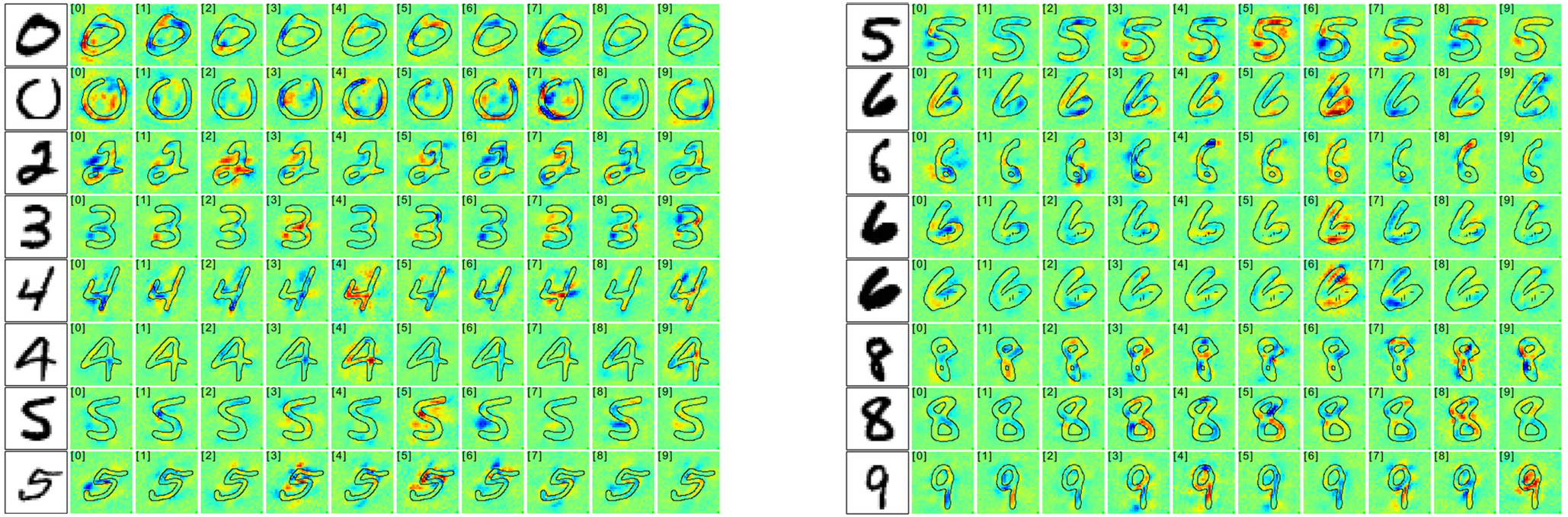


Fig 17. Pixel-wise decompositions for all classes for 16 randomly drawn digits from the MNIST test set. Results are obtained using the relevance propagation Formula (56) with the rectifying network trained for 1 000 000 iterations from Section *MNIST experiments I I*.

Python libraries & Github

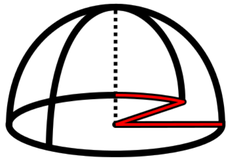
DeepLift – keras, tensorflow

lrp_toolbox

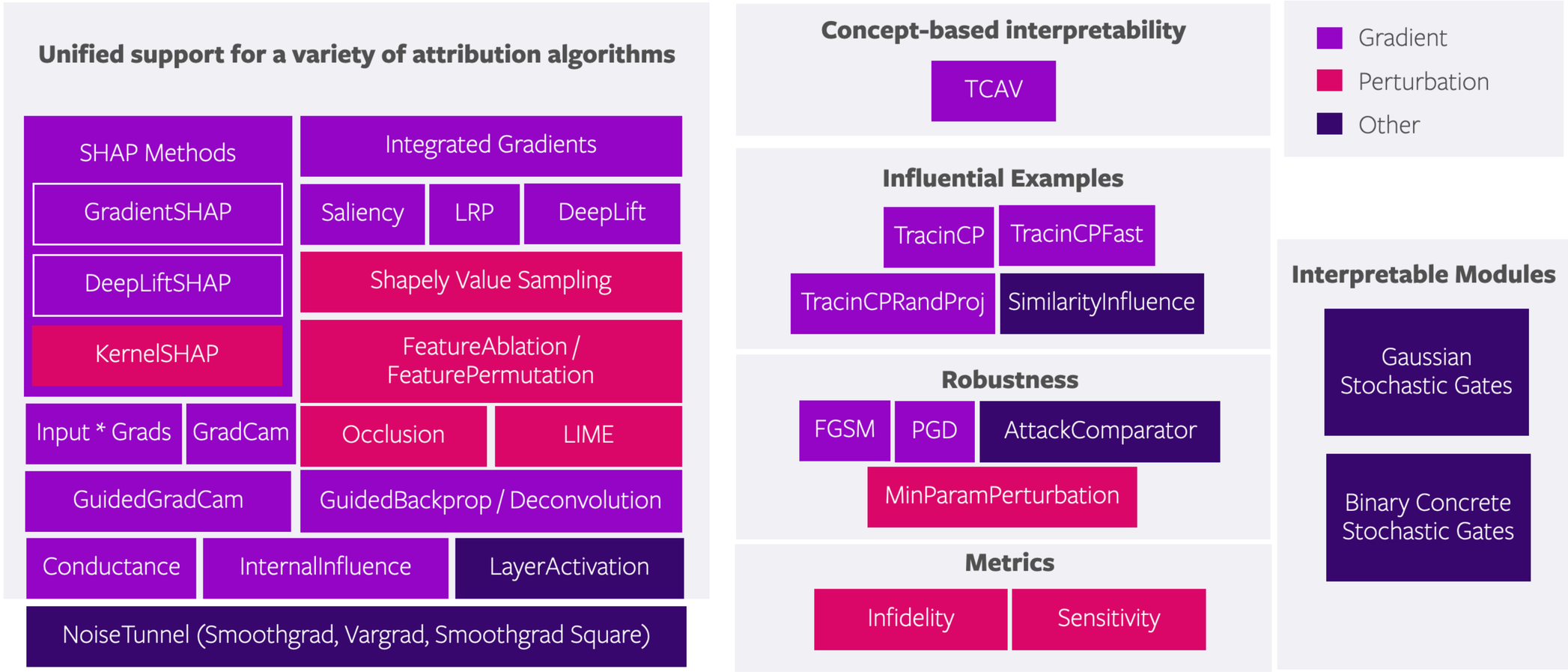
Innvestigate – keras, tensorflow

Zennit - pytorch

Captum - pytorch



Captum


Unified support for a variety of attribution algorithms
SHAP Methods
GradientSHAP
DeepLiftSHAP
KernelSHAP
Integrated Gradients
Saliency
LRP
DeepLift
Shapely Value Sampling
FeatureAblation / FeaturePermutation
Input * Grads
GradCam
Occlusion
LIME
GuidedGradCam
GuidedBackprop / Deconvolution
Conductance
InternalInfluence
LayerActivation
NoiseTunnel (Smoothgrad, Vargrad, Smoothgrad Square)
Concept-based interpretability
TCAV
Influential Examples
TracinCP
TracinCPFast
TracinCPRandProj
SimilarityInfluence
Robustness
FGSM
PGD
AttackComparator
MinParamPerturbation
Metrics
Infidelity
Sensitivity
Gradient
Perturbation
Other
Interpretable Modules
Gaussian Stochastic Gates
Binary Concrete Stochastic Gates

THIS IS THE END



Thanks for the attention!

Main References

S. Bach, A. Binder, G. Montavon, F. Klauschen, K. R. Müller, and W. Samek. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. PLoS ONE, 10(7), 2015. https://doi.org/10.1371/journal.pone.0130140.

S. M. Lundberg and S. I. Lee, A unified approach to interpreting model predictions, Advances in Neural Information Processing Systems, vol. 2017 December, pp. 4766-4775, 2017. https://dl.acm.org/doi/10.5555/3295222.3295230.

G. Montavon, S. Lapuschkin, A. Binder, W. Samek, and K. R. Müller, Explaining nonlinear classification decisions with deep taylor decomposition, Pattern Recognition, vol. 65, pp. 211–222, 2017. https://doi.org/10.1016/j.patcog.2016.11.008

G. Montavon, A. Binder, S. Lapuschkin, W. Samek, and K.R. Müller (2019). Layer-Wise Relevance Propagation: An Overview. In: Samek, W., Montavon, G., Vedaldi, A., Hansen, L., Müller, KR. (eds) Explainable AI: Interpreting, Explaining and Visualizing Deep Learning. Lecture Notes in Computer Science(), vol 11700. Springer, Cham. https://doi.org/10.1007/978-3-030-28954-6_10

W.Samek, G.Montavon, S.Lapuschkin, C.J.Anders, and K.R. Müller. Explaining deep neural networks and beyond: A review of methods and applications. Proceedings of the IEEE, 109(3):247—-278, 2021. https://doi.org/10.1109/JPROC.2021.3060483

A Shrikumar, P. Greenside, and A. Kundaje. Learning important features through propagating activation differences. 34th International Conference on Machine Learning, ICML 2017, 7:4844–4866, 2017. https://dl.acm.org/doi/10.5555/3305890.3306006.

M. Sundararajan, A. Taly, and Q. Yan. Axiomatic attribution for deep networks. 34th International Conference on Machine Learning, ICML 2017, 7:5109–5118, 2017. https://dl.acm.org/doi/10.5555/3305890.3306024.